Mini Project 01: Escape the Maze

Faizan Muhammad (BSCS25040) Hadia Khalid (BSCS25028)

Artificial Intelligence | Group 4 | 14 October 2026

1 AI Assistant Used

During the project, we used an OpenAI AI assistant to clarify search concepts, discuss approaches to the coding tasks, and work through issues encountered during development. It also helped us interpret test results and organize the report. We ran the project locally and compared its outputs with the reference results provided in the assignment.

2 Results

The tables were collected from the integrated `main` branch using `python runner.py -l <maze> -compare -timeout 10`. Greedy and A* use Manhattan distance. A node is counted as expanded when `getSuccessors` is called. Times are from one local run; values rounded to 0.000 seconds mean less than 0.0005 seconds, not zero computation time.

`maze_medium`

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	162	162	233	0.005
BFS	66	66	407	0.002
IDS	66	66	437,730	1.936
UCS	66	66	407	0.002
Greedy	80	80	81	0.000
A*	66	66	209	0.001

All six algorithms returned valid solutions.

`open_glade`

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	234	234	322	0.002
BFS	32	32	344	0.002
IDS	–	–	2,257,183	10.000
UCS	32	32	344	0.002
Greedy	36	36	39	0.000
A*	32	32	114	0.001

IDS timed out after 10 seconds; its expansion count is partial.

`ivy_maze`

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	159	88	422	0.004
BFS	80	60	439	0.003
IDS	–	–	2,417,801	10.000
UCS	70	64	418	0.002
Greedy	130	82	85	0.000
A*	70	64	344	0.002

IDS timed out after 10 seconds; its expansion count is partial.

3 DFS vs BFS

On `maze_medium`, DFS returns 162 steps while BFS returns 66. DFS follows the most recently pushed successor and pursues that branch before alternatives. It stops at its first goal, so a winding route can be found before a much shorter one. BFS uses a FIFO queue and explores paths in increasing depth, finding a goal with the fewest moves. DFS expands fewer states here (233 versus 407), but that does not make its solution better.

Both implementations use an expanded set and are complete on the finite maze state space: without a runtime limit, they find a reachable goal or exhaust the reachable states. DFS is not generally complete in an infinite search space and is not optimal. BFS is complete with finite branching and a finite-depth solution; it minimizes cost when all moves have the same positive cost. With different terrain costs it still minimizes steps, but need not minimize total cost.

4 Iterative Deepening Search

IDS runs DLS with limits 0, 1, 2, and so on. It continues only after `CUTOFF`, and stops on a solution or `None`. At the depth boundary, DLS tests for a goal but does not expand the state.

Our DLS prevents cycles by tracking only the states on the current path. On backtracking, a state is removed from that set. A global explored set would be incorrect: reaching a state first through a long route must not prevent a later, shorter route from exploring it with more depth budget remaining.

The narrow, non-looping corridors of `maze_small` offer relatively few alternative simple paths. IDS therefore finds its 50-step solution after 1,352 expansions. The open areas and loops of `open_glade` create many different simple paths to the same position. Path-based cycle detection stops a loop within one path, but permits reaching that position through other paths. IDS repeats this work at each new limit. It expands over 2.25 million nodes without finishing in our 10-second run, whereas BFS finds the 32-step solution after 344 expansions.

A timeout does not prove that a maze is unsolvable. Timing and the number of expansions completed before a timeout depend on the machine.

In exchange for repeated work, IDS retains a depth-first search frontier rather than BFS's entire breadth frontier. Its memory demand grows mainly with search depth instead of the width of the deepest layer; our copied action lists add some path-storage overhead. With sufficient maximum depth, finite branching, and no timeout, IDS finds a shallowest solution. Like BFS, this is cost-optimal only when move costs are equal.

5 Path Costs: BFS vs UCS

On `griever_alley`, BFS chooses 20 steps with cost 56; UCS chooses 28 steps with cost 42. BFS minimizes the number of actions, so it takes the shorter route through expensive Griever territory. UCS orders its priority queue by accumulated cost $g(n)$, allowing a longer detour when its total cost is lower. The cost is charged on entering the destination cell: corridor 1, ivy 3, and Griever territory 10.

BFS and UCS are guaranteed to return equally good costs when every legal move has the same positive cost, as in a uniform-cost corridor maze. They may choose different routes of equal quality. With mixed terrain there is no such guarantee, although their answers can coincide. In `ivy_maze`, BFS gives 60 steps at cost 80, while UCS gives 64 steps at cost 70.

6 Heuristics

Why greedy makes a poor choice. Greedy chooses the state that looks closest to the exit using only $h(n)$. It ignores how much the route has already cost. On `greedy_trap`, this leads it through expensive terrain: its route has 25 steps but costs 52. A* considers both the cost already paid and the estimated cost still remaining, using $f(n) = g(n) + h(n)$. It finds a different 25-step route costing only 25. Greedy expands 60 states, while A* expands 155; fewer expansions do not always mean a better route.

Is Euclidean distance admissible? Yes. Euclidean distance measures the straight-line distance to the exit. The runner can only move horizontally or vertically, and walls may require detours. Therefore, the actual route cannot be shorter than the straight line. Every move also costs at least 1, so this distance cannot overestimate the cheapest remaining cost. Both Euclidean and Manhattan distance are consistent: one move changes either estimate by at most 1, no more than the move cost.

A* on `maze_big`	Cost	Steps	Expanded
Manhattan	124	124	679
Euclidean	124	124	880

Why Manhattan expands fewer states. Manhattan distance adds the horizontal and vertical distances. This matches the runner's four-direction movement better than the straight-line estimate. It is at least as large as Euclidean distance, but still does not overestimate. In this test, its stronger guidance lets A* expand 201 fewer states. Both heuristics return the same optimal cost of 124. Neither accounts for walls, so both can underestimate the real remaining cost.

7 Q9: Key Hunt Heuristic

What the heuristic estimates. The runner must collect all remaining keys and then reach the exit. Our state stores the current position and a `frozenset` of remaining keys. Using shortest maze distances in steps, the heuristic adds three estimates:

1. Distance from the current position to the nearest remaining key.
2. Minimum total distance needed to connect all remaining keys.
3. Distance from the key nearest the exit to the exit.

The second estimate uses a *minimum spanning tree* (MST): a set of connections joining all remaining keys with the smallest possible total distance and no cycles. The tree is a lower bound, not the route the runner will follow. With one remaining key its tree cost is zero. With no remaining keys, the heuristic is simply the distance to the exit. We use the provided `mazeDistance` helper and cache distances and tree costs to avoid repeated calculations.

Why it does not overestimate. Any successful journey has three separate parts: reaching the first key, travelling between keys until all are collected, and reaching the exit from the last key. The first and last parts are at least their nearest-key estimates. The middle part connects every remaining key and cannot be shorter than the MST. Adding these bounds is valid because they refer to separate parts of the journey. Every move costs at least 1, so a bound measured in steps also bounds the true terrain cost.

Why it is consistent. Consistency requires the estimate before a move to be no greater than the move cost plus the estimate afterwards:

$$h(\text{current}) \leq \text{move cost} + h(\text{next}).$$

There are two cases.

No key is collected. The remaining-key set does not change, so the MST and key-to-exit estimates stay the same. Moving one cell can reduce the nearest-key distance by at most one step. Since the move costs at least 1, the inequality holds. When no keys remain, the same argument applies to distance to the exit.

A key is collected. The old nearest-key distance is at most the cost of the move onto that key. If keys remain, reconnecting the collected key to the new MST costs its distance to the nearest remaining key. This gives a tree for the old key set, whose weight cannot be less than the old minimum tree weight. The key-to-exit estimate cannot decrease when a key is removed. Together, these bounds give the required inequality. If the last key is collected, only its distance to the exit remains. At the goal all keys have been collected and the runner is at the exit, so the heuristic is zero.

Results and verification. On `keys_medium`, the starting estimate is 83. A* finds the optimal 94-step route with cost 94 after **446 expansions**, compared with 9,695 for UCS. This is approximately 95.4% fewer expansions and is below the assignment's 1,200-expansion threshold. On `keys_tiny` and `keys_small`, A* returns costs 26 and 54 with 93 and 149 expansions; all costs match UCS.

We checked consistency over every reachable transition in the three supplied key mazes, including all 11,004 states and 24,206 transitions in `keys_medium`. Goal values were zero. Weighted-terrain and no-key tests also passed. The integrated code passed all 16 reference cases and 77 additional search runs across the 11 provided layouts.